

05_noise_robustness

June 24, 2026

1 Step 5: Robustness, Sensitivity, Corruption, and Exploratory Conformal Diagnostics (Authoritative v4)

This notebook keeps the broader repository-facing Step 5 diagnostic surface intact while making the framing strict. Step 5 is a **diagnostic stress-testing layer** built on the frozen v4 models and predictions. Its role is to sharpen interpretation of the saved models, not to be overstated as a deployment-realism guarantee.

```
[1]: import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from IPython.display import display

def find_repo_root() -> Path:
    candidates = [Path.cwd().resolve(), *Path.cwd().resolve().parents]
    for base in candidates:
        if (base / 'data' / 'processed' / 'v4' / 'v4_protocol_manifest.json').
            exists():
                return base
    raise RuntimeError('Could not locate repository root from the current_
working directory.')

REPO_ROOT = find_repo_root()
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))
if str(REPO_ROOT / 'src') not in sys.path:
    sys.path.insert(0, str(REPO_ROOT / 'src'))
STEP_TAG = 'step5'
FIG_PDF = REPO_ROOT / 'report' / 'figures' / 'v4' / 'pdf' / STEP_TAG
FIG_PNG = REPO_ROOT / 'report' / 'figures' / 'v4' / 'png' / STEP_TAG
```

```

TABLE_DIR = REPO_ROOT / 'report' / 'tables' / 'v4'
for directory in (FIG_PDF, FIG_PNG, TABLE_DIR):
    directory.mkdir(parents=True, exist_ok=True)

plt.rcParams.update({
    'font.family': 'DejaVu Serif',
    'font.size': 9,
    'axes.labelsize': 9,
    'axes.titlesize': 9,
    'xtick.labelsize': 8,
    'ytick.labelsize': 8,
    'legend.fontsize': 7,
    'figure.dpi': 180,
    'savefig.bbox': 'tight',
    'pdf.fonttype': 42,
    'ps.fonttype': 42,
})

IEEE_ONE_COL = 3.5
IEEE_TWO_COL = 7.16
COND_ORDER = ['pd', 'hd', 'als']
COND_LABEL = {'pd': 'PD', 'hd': 'HD', 'als': 'ALS'}
COND_COLOR = {'pd': '#4C78A8', 'hd': '#F58518', 'als': '#54A24B'}
CLF_ORDER = ['rf', 'knn', 'svm', 'dt', 'qda', 'xgb', 'lgbm']
CLF_LABEL = {'rf': 'RF', 'knn': 'KNN', 'svm': 'SVM', 'dt': 'DT', 'qda': 'QDA',
    ↪ 'xgb': 'XGBoost', 'lgbm': 'LightGBM'}
DIRECTIONS = ['pd_to_hd', 'hd_to_pd', 'pd_to_als', 'als_to_pd', 'hd_to_als',
    ↪ 'als_to_hd']
DIR_LABEL = {d: d.replace('_to_', '→').upper() for d in DIRECTIONS}
SIGMA_KEYS = ['0.0', '0.05', '0.1', '0.15', '0.2', '0.25', '0.5']
SIGMA_LEVELS = [float(x) for x in SIGMA_KEYS]

def savefig_stem(fig: plt.Figure, stem: str) -> None:
    pdf_path = FIG_PDF / f'{stem}.pdf'
    png_path = FIG_PNG / f'{stem}.png'
    fig.savefig(pdf_path)
    fig.savefig(png_path, dpi=300)
    print(f'Saved figure: {pdf_path.relative_to(REPO_ROOT)}')
    print(f'Saved figure: {png_path.relative_to(REPO_ROOT)}')

def save_table_csv(df: pd.DataFrame, name: str, *, index: bool = False) -> None:
    path = TABLE_DIR / name
    df.to_csv(path, index=index)
    print(f'Saved table: {path.relative_to(REPO_ROOT)}')

```

```

def subject_coverage_mean(value):
    if isinstance(value, dict):
        return float(value['mean_subject_coverage'])
    return float(value)

BASE = REPO_ROOT / 'experiments' / 'results' / 'v4'
noise = json.loads((BASE / 'noise_robustness_v4.json').read_text())['data']
feature_sens = json.loads((BASE / 'feature_sensitivity_v4.json').
    ↳read_text())['data']
subject_sens = json.loads((BASE / 'subject_sensitivity_v4.json').
    ↳read_text())['data']
corruption = json.loads((BASE / 'corruption_robustness_v4.json').
    ↳read_text())['data']
conformal = json.loads((BASE / 'conformal_v4.json').read_text())['data']

```

1.1 Gaussian feature-space stress test

The within- and cross-condition severity curves are preserved as the primary robustness view because they summarize how performance responds to controlled feature-space perturbation. A clear within-condition v4 pattern is that **ALS remains the most resilient condition at heavier stress levels** (mean $f1@0.50 = 0.7921$), whereas **HD degrades the fastest** (mean $f1@0.50 = 0.6599$). The earliest 10% drops also cluster in HD, with some HD classifiers crossing that threshold by $\sigma=0.10$ - 0.20 . Under cross-condition stress, the direction-level curves remain heterogeneous, which is one reason this section stays diagnostic rather than predictive of deployment behavior.

```

[2]: noise_rows = []
for condition in COND_ORDER:
    for clf_name in CLF_ORDER:
        if clf_name not in noise['within'][condition]:
            continue
        for sigma_key, sigma in zip(SIGMA_KEYS, SIGMA_LEVELS):
            values = noise['within'][condition][clf_name][sigma_key]
            noise_rows.append({
                'scope': 'within',
                'domain': condition,
                'classifier': clf_name,
                'variant': 'direct',
                'sigma': sigma,
                'mean_f1': float(np.mean(values)),
                'std_f1': float(np.std(values, ddof=1)) if len(values) > 1 else_
↳0.0,
            })
for direction in DIRECTIONS:

```

```

    for relative_scope in ['pooled_relative', 'source_relative',
↳ 'target_relative']:
        for clf_name in CLF_ORDER:
            if clf_name not in noise['cross'][direction][relative_scope]:
                continue
            for sigma_key, sigma in zip(SIGMA_KEYS, SIGMA_LEVELS):
                values =
↳ noise['cross'][direction][relative_scope][clf_name][sigma_key]
                noise_rows.append({
                    'scope': 'cross',
                    'domain': direction,
                    'classifier': clf_name,
                    'variant': relative_scope,
                    'sigma': sigma,
                    'mean_f1': float(np.mean(values)),
                    'std_f1': float(np.std(values, ddof=1)) if len(values) > 1
↳ else 0.0,
                })
noise_df = pd.DataFrame(noise_rows)
save_table_csv(noise_df, 'step5_noise_summary_v4.csv')
clf_palette = dict(zip(CLF_ORDER, sns.color_palette('tab10',
↳ n_colors=len(CLF_ORDER))))

fig, axes = plt.subplots(3, 7, figsize=(13.0, 5.6), sharex=True, sharey=True,
↳ constrained_layout=True)
for i, cond in enumerate(COND_ORDER):
    for j, clf in enumerate(CLF_ORDER):
        ax = axes[i, j]
        frame = noise_df[(noise_df['scope'] == 'within') & (noise_df['domain']
↳ == cond) & (noise_df['classifier'] == clf)].sort_values('sigma')
        ax.plot(frame['sigma'], frame['mean_f1'], marker='o', linewidth=1.1,
↳ color=COND_COLOR[cond])
        ax.fill_between(frame['sigma'], frame['mean_f1'] - frame['std_f1'],
↳ frame['mean_f1'] + frame['std_f1'], alpha=0.18, color=COND_COLOR[cond])
        if i == 0:
            ax.set_title(CLF_LABEL[clf], fontsize=8)
        if j == 0:
            ax.set_ylabel(COND_LABEL[cond])
            ax.set_ylim(0.0, 1.02)
            ax.grid(alpha=0.18)
savefig_stem(fig, 'noise_curves_within')
plt.show()

fig, axes = plt.subplots(2, 3, figsize=(IEEE_TWO_COL, 6.45), sharex=True,
↳ sharey=True, gridspec_kw={'hspace': 0.20, 'wspace': 0.24})
axes = axes.ravel()

```

```

for ax, direction in zip(axes, DIRECTIONS):
    for clf in CLF_ORDER:
        frame = noise_df[(noise_df['scope'] == 'cross') & (noise_df['domain'] ==
        direction) & (noise_df['variant'] == 'pooled_relative') &
        (noise_df['classifier'] == clf)].sort_values('sigma')
        ax.plot(frame['sigma'], frame['mean_f1'], linewidth=0.95, alpha=0.85,
        color=clf_palette[clf], label=CLF_LABEL[clf])
        ax.set_title(DIR_LABEL[direction])
        ax.set_ylim(0.0, 1.02)
        ax.grid(alpha=0.18)
handles = [plt.Line2D([0], [0], color=clf_palette[clf], lw=1.8,
        label=CLF_LABEL[clf]) for clf in CLF_ORDER]
fig.legend(handles, [CLF_LABEL[clf] for clf in CLF_ORDER], loc='lower center',
        bbox_to_anchor=(0.5, 0.12), ncol=4, frameon=False)
fig.subplots_adjust(bottom=0.24, top=0.95)
savefig_stem(fig, 'noise_curves_cross')
plt.show()

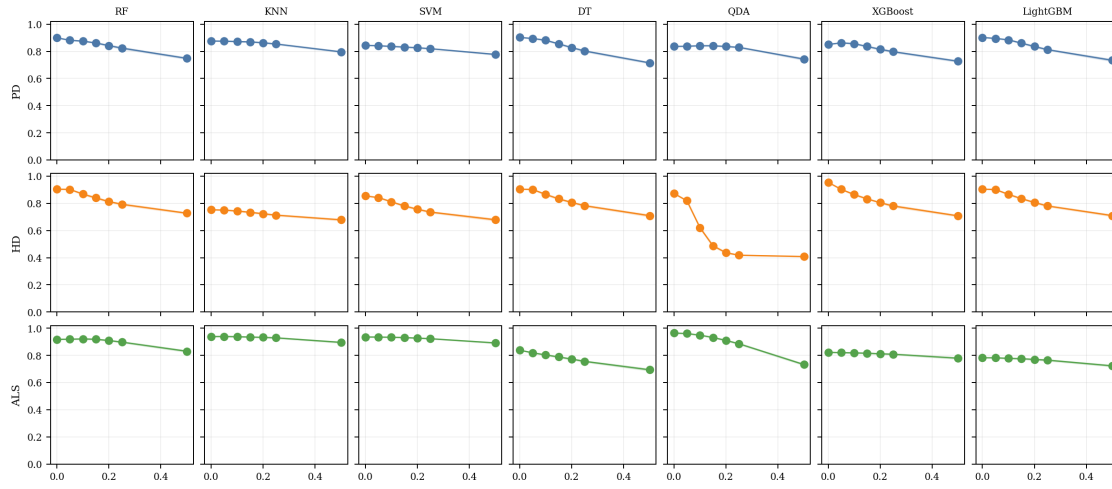
fig, axes = plt.subplots(2, 3, figsize=(IEEE_TWO_COL, 5.8), sharex=True,
        sharey=True, constrained_layout=True)
fig.set_constrained_layout_pads(hspace=0.10, wspace=0.02)
axes = axes.ravel()
for ax, direction in zip(axes, DIRECTIONS):
    for variant, style, color in [
        ('pooled_relative', '-', '#4C78A8'),
        ('source_relative', '--', '#F58518'),
        ('target_relative', ':', '#54A24B'),
    ]:
        frame = (
            noise_df[(noise_df['scope'] == 'cross') & (noise_df['domain'] ==
            direction) & (noise_df['variant'] == variant)]
            .groupby('sigma', as_index=False)['mean_f1'].mean()
            .sort_values('sigma')
        )
        ax.plot(frame['sigma'], frame['mean_f1'], linestyle=style, marker='o',
        linewidth=1.0, color=color, label=variant)
        ax.set_title(DIR_LABEL[direction])
        ax.set_ylim(0.0, 1.02)
        ax.grid(alpha=0.18)
handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', bbox_to_anchor=(0.5, 1.05),
        ncol=3, frameon=False)
savefig_stem(fig, 'step5_cross_noise_variants')
plt.show()

```

Saved table: report/tables/v4/step5_noise_summary_v4.csv

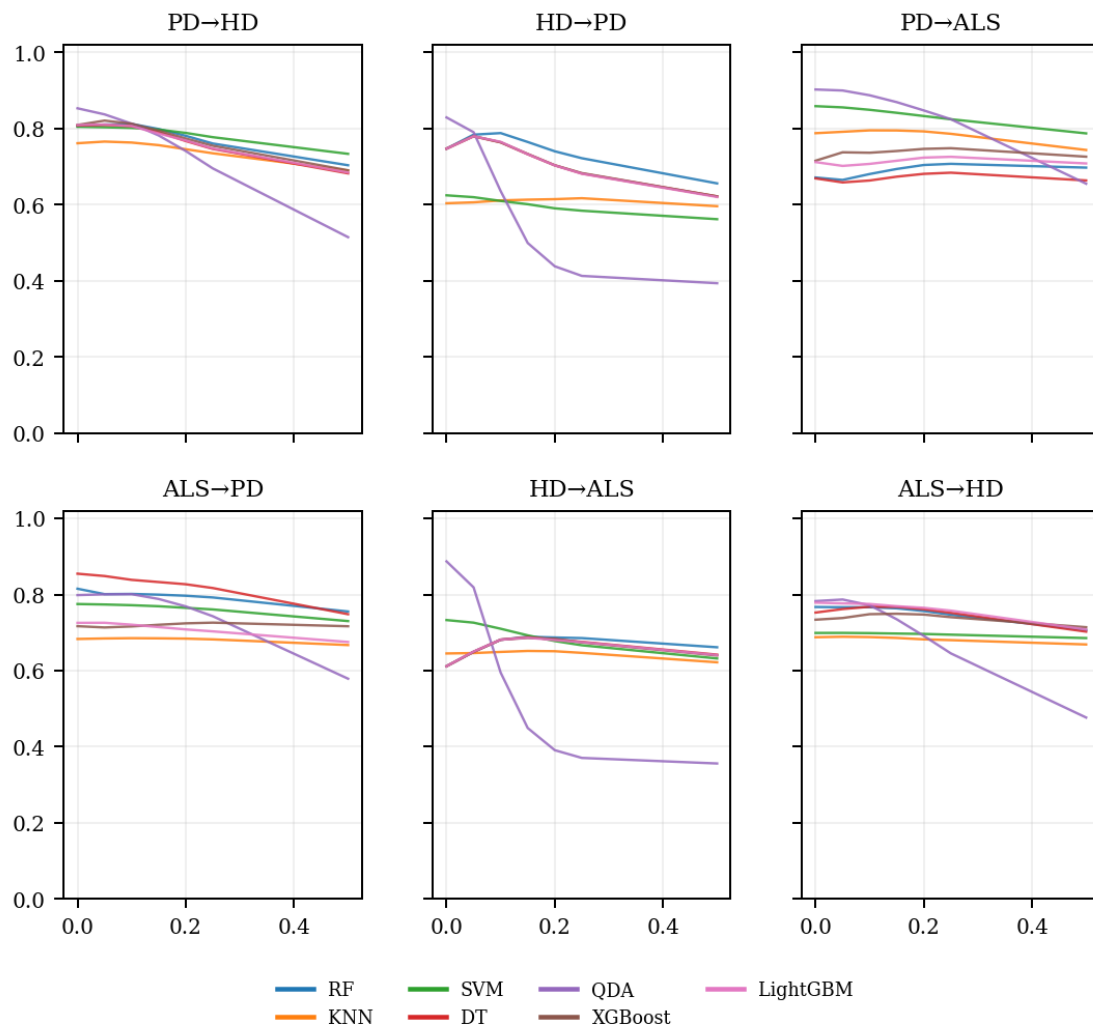
Saved figure: report/figures/v4/pdf/step5/noise_curves_within.pdf

Saved figure: report/figures/v4/png/step5/noise_curves_within.png



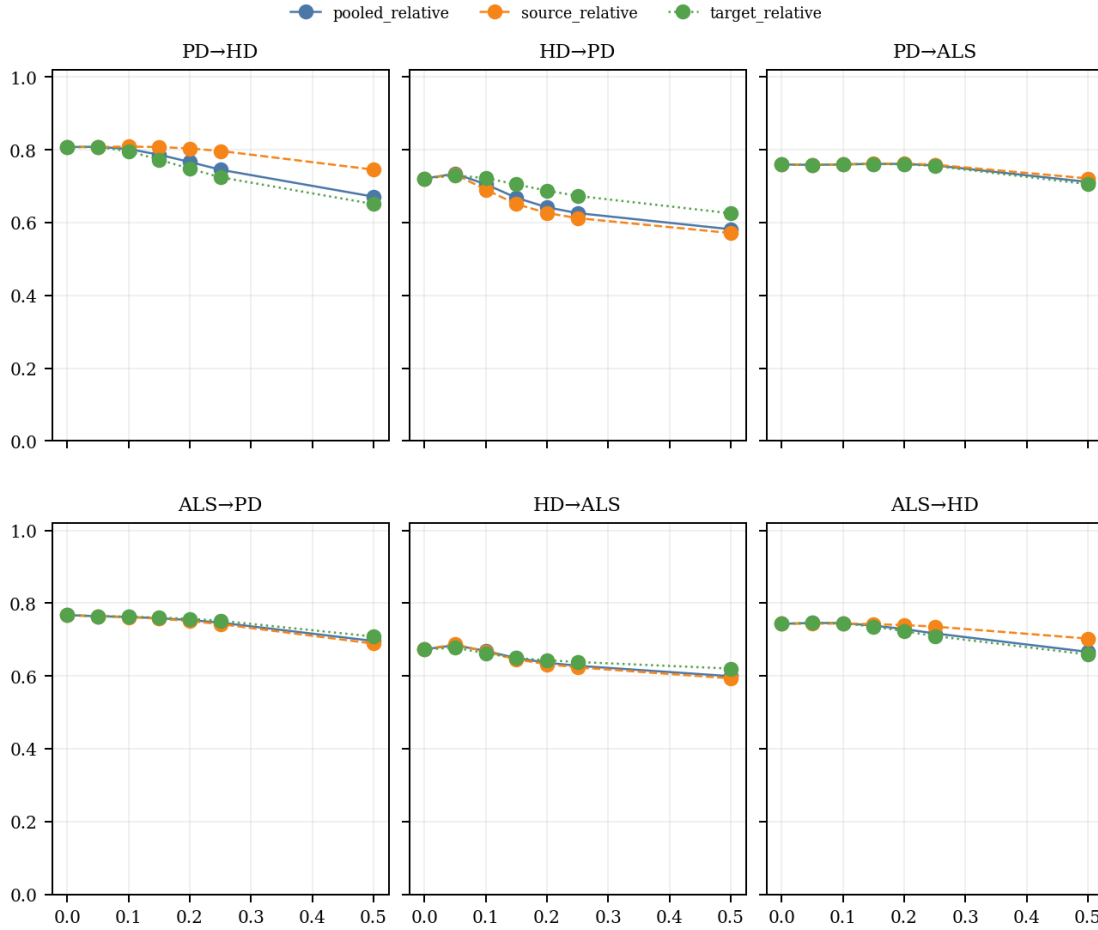
Saved figure: report/figures/v4/pdf/step5/noise_curves_cross.pdf

Saved figure: report/figures/v4/png/step5/noise_curves_cross.png



Saved figure: report/figures/v4/pdf/step5/step5_cross_noise_variants.pdf

Saved figure: report/figures/v4/png/step5/step5_cross_noise_variants.png



1.2 Noise-robustness summary table

This table keeps sigma-zero replay and key severity checkpoints visible without forcing them into one crowded plot. The sigma-zero rows confirm that the perturbation pipeline collapses back to the frozen authoritative baselines when no stress is applied. The later checkpoints show which models hold their ground and which lose performance early. The safest manuscript use is descriptive: Step 5 compares perturbation sensitivity across conditions and transfer settings; it does **not** validate real-world sensor robustness.

```
[3]: rows = []
for condition in COND_ORDER:
    for clf_name in CLF_ORDER:
        frame = noise_df[(noise_df['scope'] == 'within') & (noise_df['domain'] ==
        == condition) & (noise_df['classifier'] == clf_name)].sort_values('sigma')
        getm = lambda sigma: float(frame[frame['sigma'] == sigma]['mean_f1'].
        iloc[0])
        f0, f10, f25, f50 = getm(0.0), getm(0.1), getm(0.25), getm(0.5)
```

```

        threshold = 0.9 * f0
        sigma_drop = next((sigma for sigma in SIGMA_LEVELS if getm(sigma) <=
↳threshold), np.nan)
        rows.append({
            'scope': f'within:{condition}',
            'classifier': clf_name,
            'f1@0': f0,
            'f1@0.10': f10,
            'f1@0.25': f25,
            'f1@0.50': f50,
            'delta_f1@0.25': f0 - f25,
            'sigma_at_10pct_drop': sigma_drop,
        })
summary_df = pd.DataFrame(rows)
save_table_csv(summary_df, 'step5_noise_robustness_summary_v4.csv')
display(
    summary_df.style
        .format({
            'f1@0': '{:.4f}',
            'f1@0.10': '{:.4f}',
            'f1@0.25': '{:.4f}',
            'f1@0.50': '{:.4f}',
            'delta_f1@0.25': '{:+.4f}',
            'sigma_at_10pct_drop': '{:.2f}',
        })
        .background_gradient(subset=['delta_f1@0.25'], cmap=sns.
↳light_palette('#8B1E3F', as_cmap=True), vmin=0.0, vmax=0.35)
        .set_caption('Within-condition gaussian_feature_space_stress_test summary')
)

```

Saved table: report/tables/v4/step5_noise_robustness_summary_v4.csv

<pandas.io.formats.style.Styler at 0x7fc4b35948f0>

1.3 Feature sensitivity: per-feature heatmaps, family summaries, and top recurring drops

The annotated heatmaps are retained because they show both marginal single-feature sensitivity and direction-wise variation in those drops. The family summary is especially important for the final paper because it aligns with Step 4: across cross-condition settings, **variability** produces the largest mean family-level drop (**0.1685**) and **raw timing** is next (**0.1137**), while the other families contribute smaller or more context-dependent effects. Within-condition family drops are generally smaller, which underscores that transfer amplifies sensitivity to a narrower set of timing-structure features.

```

[4]: feature_names = sorted(next(iter(feature_sens['within']['pd'].
↳values()))['per_feature'].keys())
within_feature_mat = pd.DataFrame(

```

```

[
    {
        'condition': cond,
        **{
            feature: float(np.mean([payload['per_feature'][feature] for
↳payload in feature_sens['within'][cond].values()]))
                for feature in feature_names
            },
        }
        for cond in COND_ORDER
    ]
).set_index('condition')

cross_feature_mat = pd.DataFrame(
    [
        {
            'direction': direction,
            **{
                feature: float(np.mean([payload['per_feature'][feature] for
↳payload in feature_sens['cross'][direction].values()]))
                    for feature in feature_names
                },
            }
            for direction in DIRECTIONS
        ]
    ).set_index('direction')

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.65, 4.7),
↳constrained_layout=True)
sns.heatmap(
    within_feature_mat.rename(index=COND_LABEL).T,
    cmap=sns.light_palette('#0B3C5D', as_cmap=True),
    annot=True,
    fmt='.3f',
    annot_kws={'size': 7},
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Mean marginal F1 drop'},
    ax=ax,
)
ax.set_title('Within-condition engineered_feature_permutation_sensitivity')
ax.set_xlabel('Condition')
ax.set_ylabel('Feature')
savefig_stem(fig, 'step5_feature_sensitivity_within')
plt.show()

```

```

fig, ax = plt.subplots(figsize=(IEEE_TWO_COL * 1.05, 4.95),
    ↳constrained_layout=True)
sns.heatmap(
    cross_feature_mat.rename(index=DIR_LABEL).T,
    cmap=sns.light_palette('#0B3C5D', as_cmap=True),
    annot=True,
    fmt='.3f',
    annot_kws={'size': 6.7},
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Mean marginal F1 drop'},
    ax=ax,
)
ax.set_title('Cross-condition engineered_feature_permutation_sensitivity')
ax.set_xlabel('Transfer direction')
ax.set_ylabel('Feature')
ax.set_xticklabels(ax.get_xticklabels(), rotation=24, ha='right')
savefig_stem(fig, 'step5_feature_sensitivity_cross')
plt.show()

family_rows = []
top_rows = []
for scope_name, scope_payload in feature_sens.items():
    for domain_key, domain_payload in scope_payload.items():
        for clf_name, clf_payload in domain_payload.items():
            for family, value in clf_payload['per_family'].items():
                family_rows.append({'scope': scope_name, 'domain': domain_key,
    ↳'classifier': clf_name, 'family': family, 'drop': float(value)})
                ranked = pd.Series(clf_payload['per_feature']).
    ↳sort_values(ascending=False).head(3)
                for rank, (feature, value) in enumerate(ranked.items(), start=1):
                    top_rows.append({'scope': scope_name, 'domain': domain_key,
    ↳'classifier': clf_name, 'rank': rank, 'feature': feature, 'drop':
    ↳float(value)})
family_df = pd.DataFrame(family_rows)
top_df = pd.DataFrame(top_rows)
save_table_csv(family_df, 'step5_feature_family_sensitivity_v4.csv')
save_table_csv(top_df, 'step5_feature_top3_v4.csv')

family_plot = family_df.groupby(['scope', 'family'], as_index=False)['drop'].
    ↳mean()
fig, axes = plt.subplots(1, 2, figsize=(IEEE_TWO_COL, 3.0), sharey=True,
    ↳constrained_layout=True)
for ax, scope_name in zip(axes, ['within', 'cross']):
    frame = family_plot[family_plot['scope'] == scope_name].sort_values('drop')

```

```

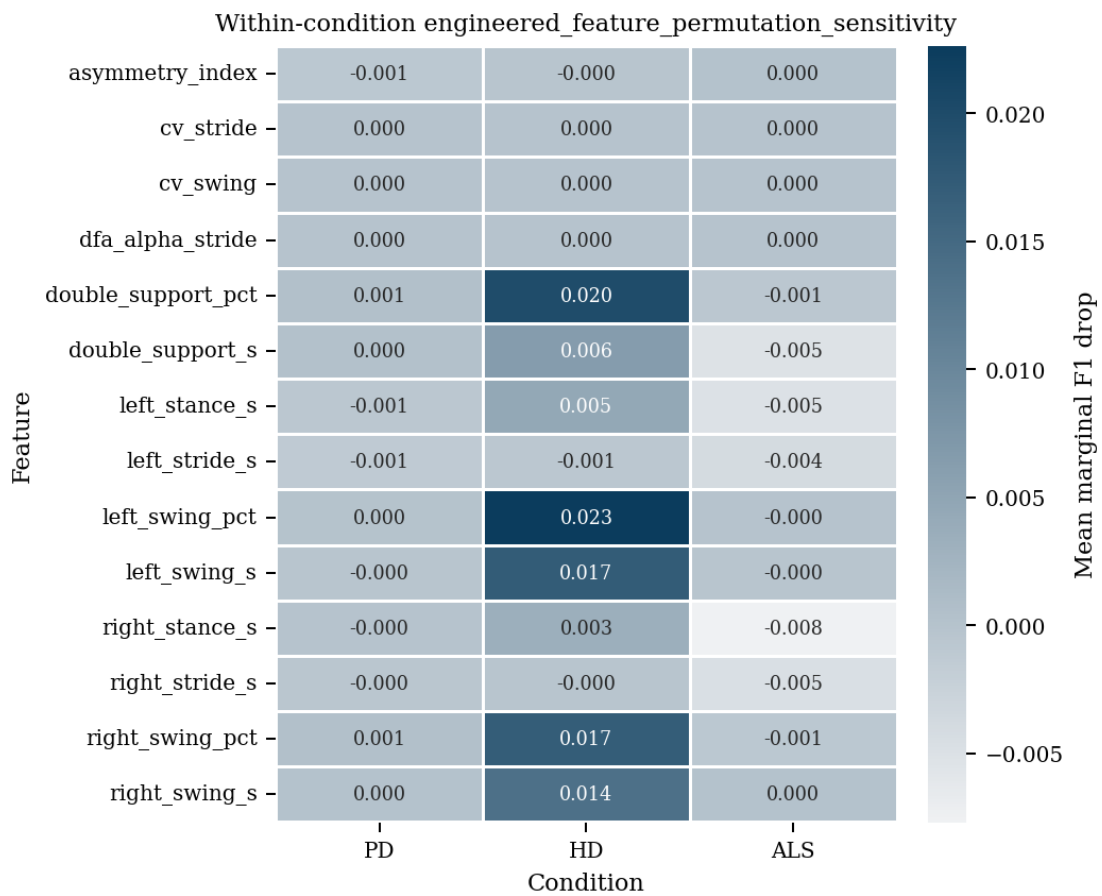
ax.barh(frame['family'], frame['drop'], color='#5DA5DA', edgecolor='black',
        linewidth=0.4)
ax.set_title(f'{scope_name.capitalize()} family sensitivity')
ax.axvline(0.0, color='black', linewidth=0.7)
ax.grid(axis='x', alpha=0.2)
savefig_stem(fig, 'step5_feature_family_sensitivity')
plt.show()

display(top_df.head(24).style.format({'drop': '{:.4f}'}).set_caption('Top three
per-feature drops by scope/domain/classifier'))

```

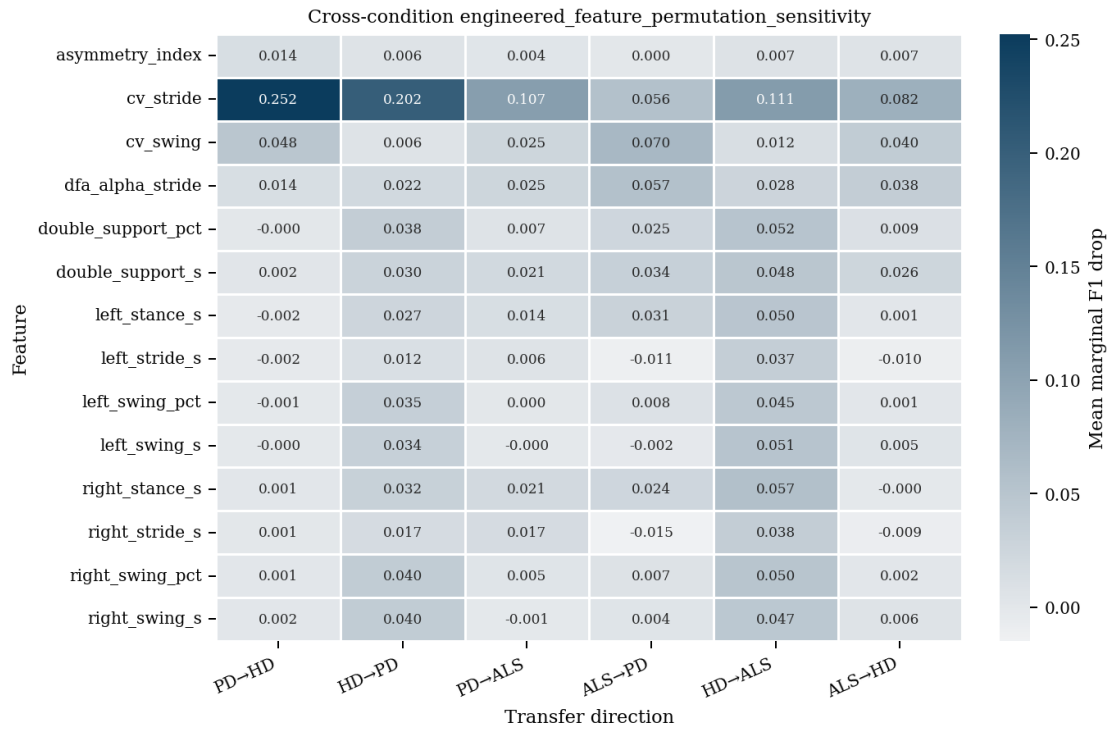
Saved figure: report/figures/v4/pdf/step5/step5_feature_sensitivity_within.pdf

Saved figure: report/figures/v4/png/step5/step5_feature_sensitivity_within.png



Saved figure: report/figures/v4/pdf/step5/step5_feature_sensitivity_cross.pdf

Saved figure: report/figures/v4/png/step5/step5_feature_sensitivity_cross.png

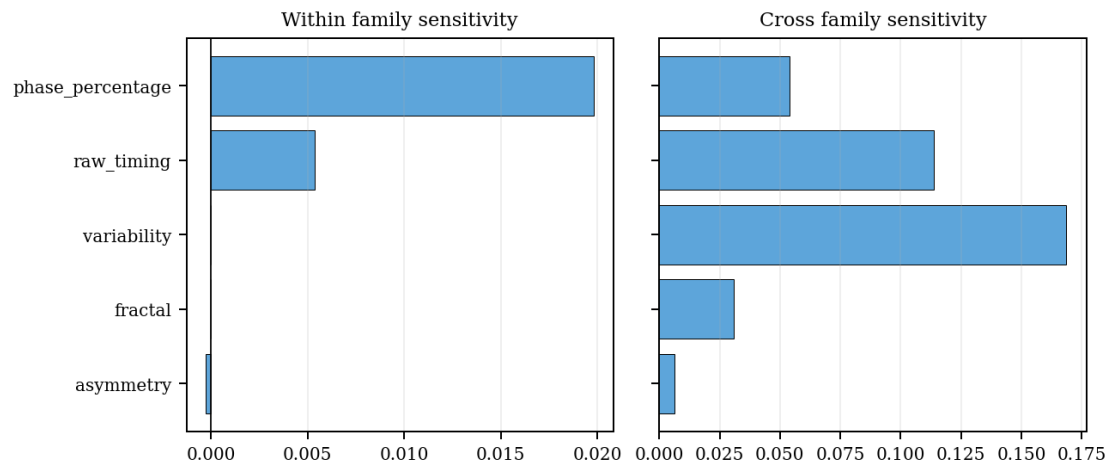


Saved table: report/tables/v4/step5_feature_family_sensitivity_v4.csv

Saved table: report/tables/v4/step5_feature_top3_v4.csv

Saved figure: report/figures/v4/pdf/step5/step5_feature_family_sensitivity.pdf

Saved figure: report/figures/v4/png/step5/step5_feature_family_sensitivity.png



<pandas.io.formats.style.Styler at 0x7fc4b6785430>

1.4 Subject sensitivity and structured corruption

Subject heterogeneity and corruption response remain separate figure families because they answer different questions. The subject-sensitivity panels show that performance is not distributed evenly across individuals, which is useful because difficult subjects can reflect genuine heterogeneity, sample size, or data quality. The corruption panels then show that stronger perturbations hurt more, with **heavy benchmark label corruption** and **heavy Gaussian feature-space jitter** among the most damaging average cross-condition stressors in the saved v4 summaries. Neither result should be oversold: difficult subjects are not automatically poor-quality subjects, and the corruption families are benchmark perturbations rather than validated operational failure models.

```
[5]: subject_rows = []
for scope_name, scope_payload in subject_sens.items():
    for domain_key, domain_payload in scope_payload.items():
        for clf_name, subject_payload in domain_payload.items():
            for subject_id, metrics in subject_payload.items():
                subject_rows.append({
                    'scope': scope_name,
                    'domain': domain_key,
                    'classifier': clf_name,
                    'subject_id': subject_id,
                    'accuracy': float(metrics['accuracy']),
                    'n_strides': int(metrics['n_strides']),
                })
subject_df = pd.DataFrame(subject_rows)
save_table_csv(subject_df, 'step5_subject_sensitivity_v4.csv')

fig, axes = plt.subplots(1, 3, figsize=(IEEE_TWO_COL, 3.3), sharey=True,
    constrained_layout=True)
for ax, cond in zip(axes, COND_ORDER):
    frame = subject_df[(subject_df['scope'] == 'within') &
    (subject_df['domain'] == cond)].copy()
    frame['Classifier'] = frame['classifier'].map(CLF_LABEL)
    sns.boxplot(
        data=frame,
        x='Classifier',
        y='accuracy',
        color=COND_COLOR[cond],
        showfliers=False,
        linewidth=0.8,
        ax=ax,
    )
    sns.stripplot(
        data=frame,
        x='Classifier',
        y='accuracy',
        color='#1B2631',
        alpha=0.28,
```

```

        size=2.1,
        jitter=0.18,
        ax=ax,
    )
    ax.set_ylim(0.0, 1.02)
    ax.set_title(COND_LABEL[cond])
    ax.set_xlabel('Classifier')
    ax.set_ylabel('Subject accuracy' if cond == COND_ORDER[0] else '')
    ax.tick_params(axis='x', labelsiz=7)
    for label in ax.get_xticklabels():
        label.set_rotation(28)
        label.set_ha('right')
    ax.grid(axis='y', alpha=0.2)
savefig_stem(fig, 'step5_subject_sensitivity_within')
plt.show()

fig, axes = plt.subplots(3, 2, figsize=(IEEE_TWO_COL * 0.94, 8.35),
    ↳sharey=True, gridspec_kw={'hspace': 0.50, 'wspace': 0.24})
axes = axes.ravel()
for idx, (ax, direction) in enumerate(zip(axes, DIRECTIONS)):
    frame = subject_df[(subject_df['scope'] == 'cross') & (subject_df['domain']_
    ↳== direction)].copy()
    frame['Classifier'] = frame['classifier'].map(CLF_LABEL)
    sns.boxplot(
        data=frame,
        x='Classifier',
        y='accuracy',
        color=COND_COLOR[direction.split('_to_')[0]],
        showfliers=False,
        linewidth=0.8,
        ax=ax,
    )
    sns.stripplot(
        data=frame,
        x='Classifier',
        y='accuracy',
        color='#1B2631',
        alpha=0.22,
        size=1.9,
        jitter=0.18,
        ax=ax,
    )
    ax.set_ylim(0.0, 1.02)
    ax.set_title(DIR_LABEL[direction], fontsize=8.4, pad=6)
    ax.set_xlabel('Classifier')
    ax.set_ylabel('Subject accuracy' if idx % 2 == 0 else '')
    ax.tick_params(axis='x', labelsiz=7, rotation=24)

```

```

    ax.grid(axis='y', alpha=0.2)
fig.subplots_adjust(bottom=0.08, top=0.95)
savefig_stem(fig, 'step5_subject_sensitivity_cross')
plt.show()

hardest = subject_df.groupby('subject_id', as_index=False)['accuracy'].mean().
    ↪sort_values('accuracy').head(18)
fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.35, 3.6),
    ↪constrained_layout=True)
ax.barh(hardest['subject_id'], hardest['accuracy'], color='#C44E52',
    ↪edgecolor='black', linewidth=0.4)
ax.set_xlim(0, 1.0)
ax.set_title('Most difficult subjects across Step 5')
ax.grid(axis='x', alpha=0.2)
savefig_stem(fig, 'step5_hardest_subjects')
plt.show()

corruption_rows = []
for scope_name, scope_payload in corruption.items():
    for domain_key, domain_payload in scope_payload.items():
        for corruption_name, severity_payload in domain_payload.items():
            for severity_name, clf_payload in severity_payload.items():
                sample = next(iter(clf_payload.values())) if
    ↪isinstance(clf_payload, dict) and clf_payload else None
                if isinstance(sample, list):
                    for clf_name, values in clf_payload.items():
                        corruption_rows.append({
                            'scope': scope_name,
                            'domain': domain_key,
                            'corruption': corruption_name,
                            'variant': 'direct',
                            'severity': severity_name,
                            'classifier': clf_name,
                            'mean_f1': float(np.mean(values)),
                        })
                else:
                    nested_sample = next(iter(sample.values())) if
    ↪isinstance(sample, dict) and sample else None
                    if isinstance(nested_sample, list):
                        for actual_severity, nested_clf_payload in clf_payload.
    ↪items():
                            for clf_name, values in nested_clf_payload.items():
                                corruption_rows.append({
                                    'scope': scope_name,
                                    'domain': domain_key,
                                    'corruption': corruption_name,

```

```

        'variant': severity_name,
        'severity': actual_severity,
        'classifier': clf_name,
        'mean_f1': float(np.mean(values)),
    })

    else:
        raise TypeError(f'Unexpected corruption payload shape_{scope_name}/{domain_key}/{corruption_name}/{severity_name}')
    for {scope_name}/{domain_key}/{corruption_name}/{severity_name}')
corruption_df = pd.DataFrame(corruption_rows)
save_table_csv(corruption_df, 'step5_corruption_summary_v4.csv')

sev_order = ['light', 'medium', 'heavy']
cor_order = ['engineered_feature_masking',
    'engineered_feature_gain_bias_drift', 'gaussian_feature_space_jitter',
    'evaluation_row_dropout', 'benchmark_label_corruption']
cor_labels = {
    'engineered_feature_masking': 'feature\nmasking',
    'engineered_feature_gain_bias_drift': 'gain/bias\ndrift',
    'gaussian_feature_space_jitter': 'feature-space\njitter',
    'evaluation_row_dropout': 'row\ndropout',
    'benchmark_label_corruption': 'label\ncorruption',
}

within_frames = []
for cond in COND_ORDER:
    frame = (
        corruption_df[(corruption_df['scope'] == 'within') &
    'corruption_df['domain'] == cond)]
        .groupby(['corruption', 'severity'], as_index=False)['mean_f1'].mean()
    )
    mat = np.full((len(cor_order), len(sev_order)), np.nan)
    for _, row in frame.iterrows():
        mat[cor_order.index(row['corruption']), sev_order.
    index(row['severity'])] = row['mean_f1']
    within_frames.append(pd.DataFrame(mat, index=[cor_labels[c] for c in
    cor_order], columns=[s.capitalize() for s in sev_order]))
within_vmin = min(np.nanmin(frame.to_numpy(dtype=float)) for frame in
    within_frames)
within_vmax = max(np.nanmax(frame.to_numpy(dtype=float)) for frame in
    within_frames)

fig, axes = plt.subplots(1, 3, figsize=(IEEE_TWO_COL * 1.04, 4.55),
    sharey=True, constrained_layout=True)
for ax, cond, mat_df in zip(axes, COND_ORDER, within_frames):
    sns.heatmap(
        mat_df,

```

```

        cmap=sns.light_palette('#0B3C5D', as_cmap=True),
        vmin=within_vmin,
        vmax=within_vmax,
        annot=True,
        fmt='.3f',
        annot_kws={'size': 7},
        linewidths=0.45,
        linecolor='white',
        cbar=ax if axes[-1],
        cbar_kws={'label': 'Mean macro-F1'} if ax is axes[-1] else None,
        ax=ax,
    )
    ax.set_title(f'Within {COND_LABEL[cond]}')
    ax.set_xlabel('Severity')
    ax.set_ylabel('Corruption family' if ax is axes[0] else '')
    ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
savefig_stem(fig, 'step5_corruption_within')
plt.show()

cross_frames = []
for direction in DIRECTIONS:
    frame = (
        corruption_df[(corruption_df['scope'] == 'cross') &
        ↪(corruption_df['domain'] == direction)]
        .groupby(['corruption', 'severity'], as_index=False)['mean_f1'].mean()
    )
    mat = np.full((len(cor_order), len(sev_order)), np.nan)
    for _, row in frame.iterrows():
        mat[cor_order.index(row['corruption']), sev_order.
        ↪index(row['severity'])] = row['mean_f1']
    cross_frames.append(pd.DataFrame(mat, index=[cor_labels[c] for c in
    ↪cor_order], columns=[s.capitalize() for s in sev_order]))
cross_vmin = min(np.nanmin(frame.to_numpy(dtype=float)) for frame in
    ↪cross_frames)
cross_vmax = max(np.nanmax(frame.to_numpy(dtype=float)) for frame in
    ↪cross_frames)

fig, axes = plt.subplots(3, 2, figsize=(IEEE_TWO_COL * 1.0, 9.3), sharey=True,
    ↪gridspec_kw={'hspace': 0.36, 'wspace': 0.16})
axes = axes.ravel()
last_mesh = None
for idx, (ax, direction, mat_df) in enumerate(zip(axes, DIRECTIONS,
    ↪cross_frames)):
    heat = sns.heatmap(
        mat_df,
        cmap=sns.light_palette('#0B3C5D', as_cmap=True),

```

```

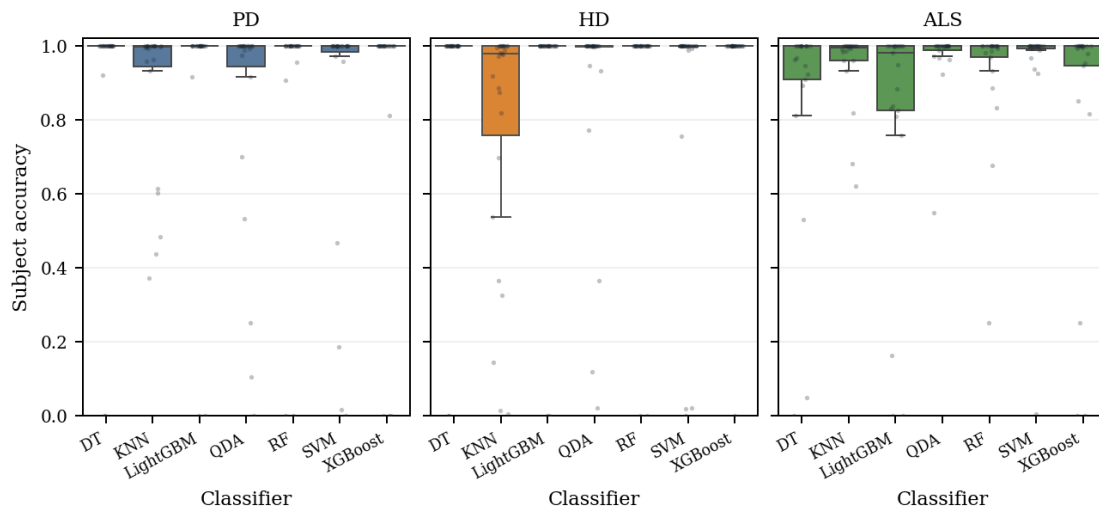
    vmin=cross_vmin,
    vmax=cross_vmax,
    annot=True,
    fmt='.3f',
    annot_kws={'size': 6.9},
    linewidths=0.45,
    linecolor='white',
    cbar=False,
    ax=ax,
)
last_mesh = heat.collections[0]
ax.set_title(DIR_LABEL[direction], fontsize=8.4, pad=6)
ax.set_xlabel('Severity')
ax.set_ylabel('Corruption family' if idx % 2 == 0 else '')
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
fig.subplots_adjust(bottom=0.14, top=0.95)
cbar_ax = fig.add_axes([0.22, 0.040, 0.56, 0.02])
cbar = fig.colorbar(last_mesh, cax=cbar_ax, orientation='horizontal')
cbar.set_label('Mean macro-F1', fontsize=8)
cbar.ax.tick_params(labelsize=7)
savefig_stem(fig, 'step5_corruption_cross')
plt.show()

```

Saved table: report/tables/v4/step5_subject_sensitivity_v4.csv

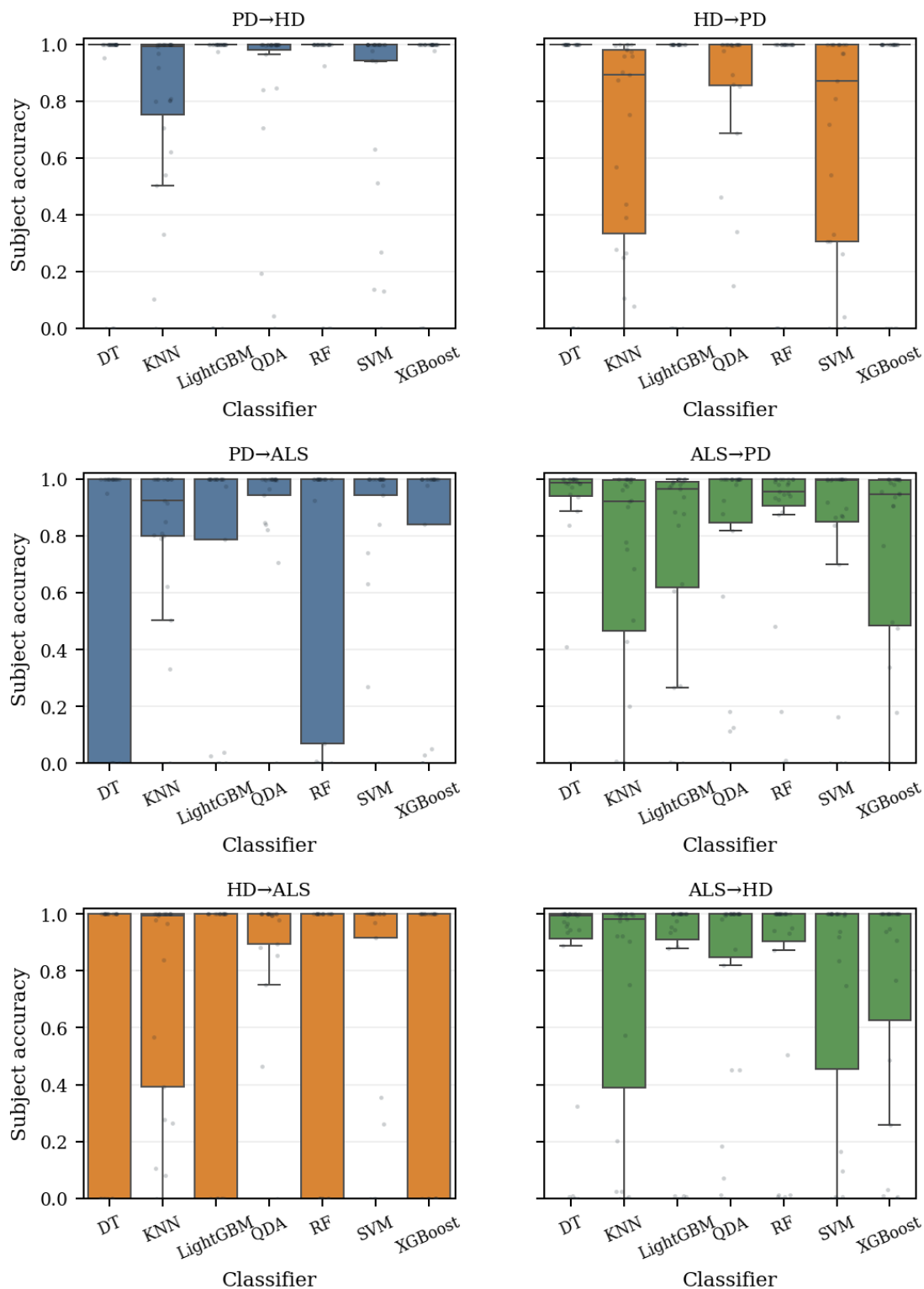
Saved figure: report/figures/v4/pdf/step5/step5_subject_sensitivity_within.pdf

Saved figure: report/figures/v4/png/step5/step5_subject_sensitivity_within.png



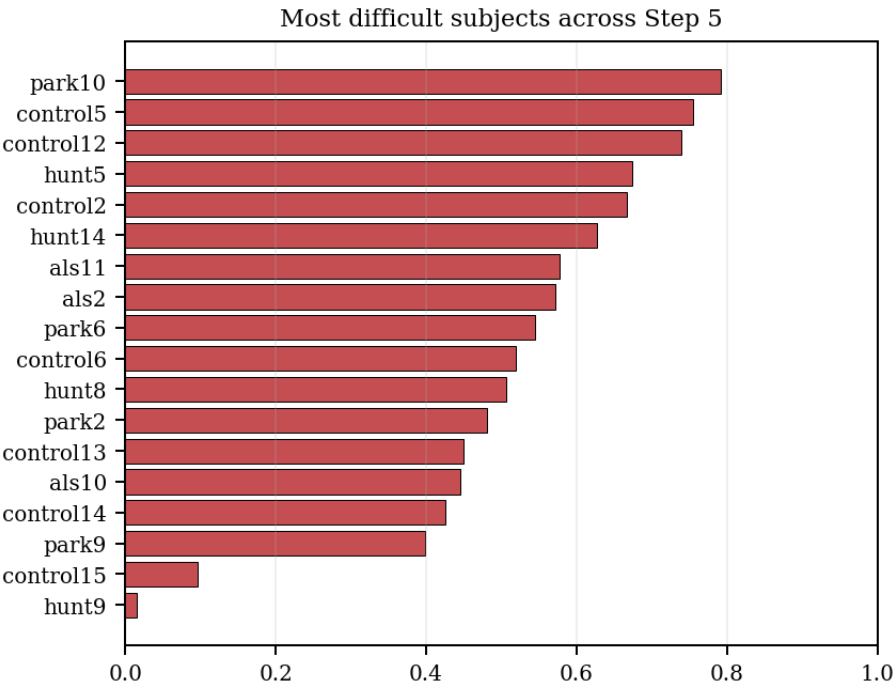
Saved figure: report/figures/v4/pdf/step5/step5_subject_sensitivity_cross.pdf

Saved figure: report/figures/v4/png/step5/step5_subject_sensitivity_cross.png



Saved figure: report/figures/v4/pdf/step5/step5_hardest_subjects.pdf

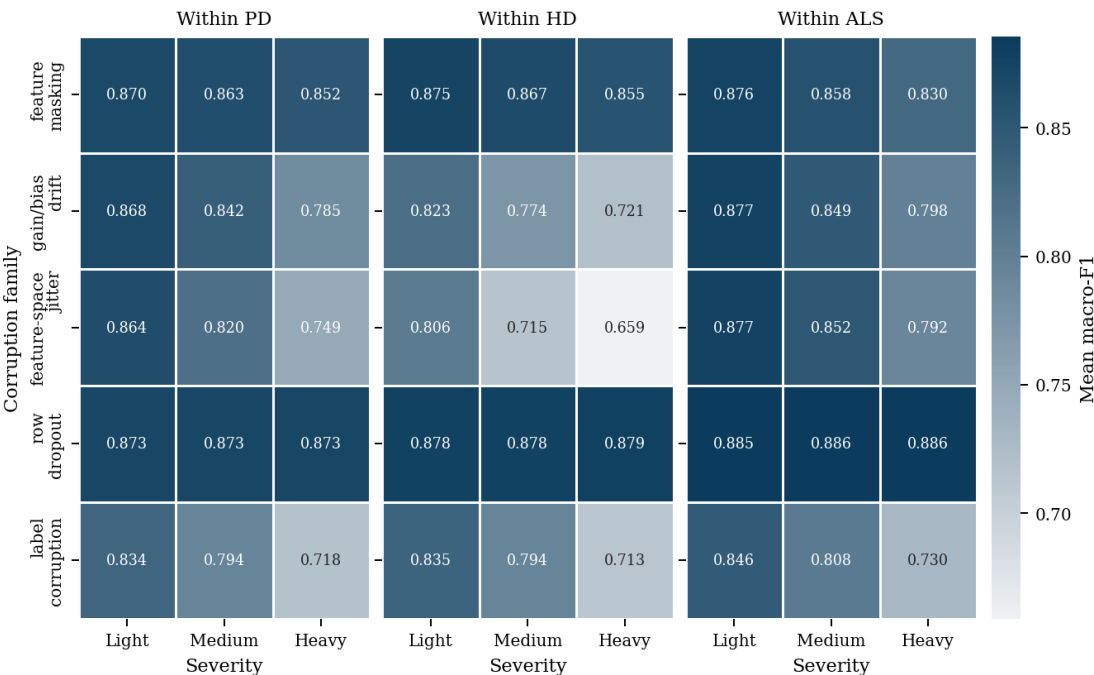
Saved figure: report/figures/v4/png/step5/step5_hardest_subjects.png



Saved table: report/tables/v4/step5_corruption_summary_v4.csv

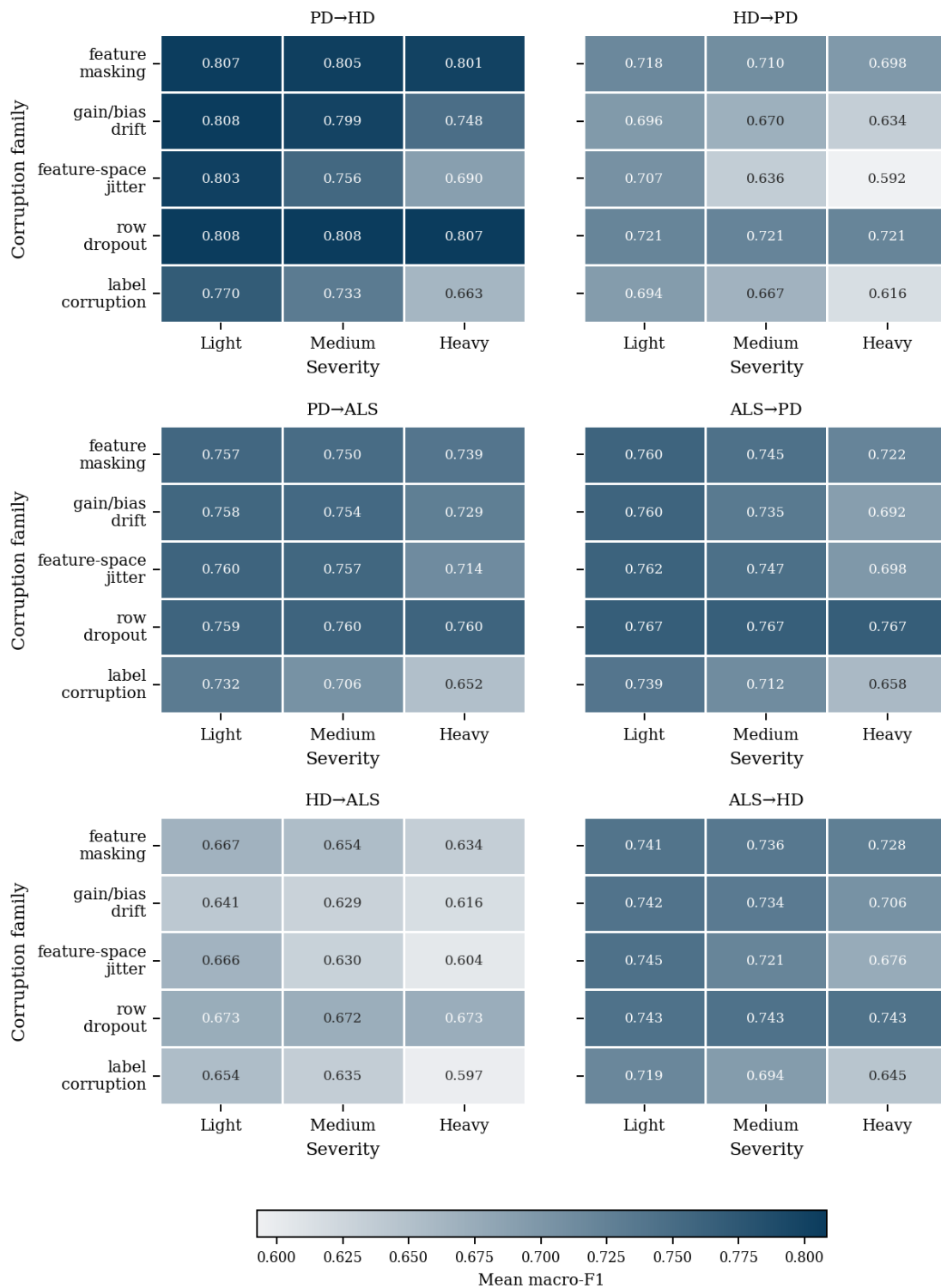
Saved figure: report/figures/v4/pdf/step5/step5_corruption_within.pdf

Saved figure: report/figures/v4/png/step5/step5_corruption_within.png



Saved figure: report/figures/v4/pdf/step5/step5_corruption_cross.pdf

Saved figure: report/figures/v4/png/step5/step5_corruption_cross.png



1.5 Exploratory conformal diagnostics

`conformal_within` uses color for condition and line style for method, while `conformal_cross` uses color for transfer direction and line style for method. The aggregate summary helps interpret those panels: within-condition **APS** keeps larger mean set sizes and higher mean subject-level coverage than **LAC**, while under cross-condition shift **LAC** becomes much tighter and more empty-set-prone before fallback (raw empty rates rising to roughly **0.23–0.34** across the tested alphas). That makes the cross-condition conformal results explicitly **exploratory under distribution shift**. They are useful for understanding set size, fallback behavior, and coverage drift, but they do not justify formal target-domain coverage guarantees.

```
[6]: coverage_rows = []
for scope_name, scope_payload in conformal.items():
    for domain_key, domain_payload in scope_payload.items():
        for clf_name, clf_payload in domain_payload.items():
            for method in ['aps', 'lac']:
                for alpha, stats in clf_payload[method].items():
                    subject_cov = stats['coverage_subject_level']
                    coverage_rows.append({
                        'scope': scope_name,
                        'domain': domain_key,
                        'classifier': clf_name,
                        'method': method,
                        'alpha': float(alpha),
                        'coverage_subject_level': subject_cov,
                        'subject_coverage_mean': subject_cov['mean_subject_coverage'],
                        'coverage_subject_min': subject_cov['min_subject_coverage'],
                        'float(subject_cov["min_subject_coverage"]) if isinstance(subject_cov, dict) else np.nan,
                        'coverage_subject_max': subject_cov['max_subject_coverage'],
                        'float(subject_cov["max_subject_coverage"]) if isinstance(subject_cov, dict) else np.nan,
                        'coverage_subject_n': float(subject_cov['n_subjects']),
                        'if isinstance(subject_cov, dict) else np.nan,
                        'coverage_marginal': float(stats['coverage_marginal']),
                        'raw_mean_set_size': float(stats['raw_set_stats']['mean_set_size']),
                        'post_mean_set_size': float(stats['post_fallback_set_stats']['mean_set_size']),
                        'raw_empty_rate': float(stats['raw_set_stats']['empty_rate']),
                        'post_empty_rate': float(stats['post_fallback_set_stats']['empty_rate']),
                    })
```

```

        for n_strides, value in clf_payload['hard_label_consensus_curve'].
        items():
            coverage_rows.append({
                'scope': scope_name,
                'domain': domain_key,
                'classifier': clf_name,
                'method': 'hard_label_consensus_curve',
                'alpha': float(n_strides),
                'coverage_subject_level': float(value),
                'coverage_subject_min': np.nan,
                'coverage_subject_max': np.nan,
                'coverage_subject_n': np.nan,
                'coverage_marginal': np.nan,
                'raw_mean_set_size': np.nan,
                'post_mean_set_size': np.nan,
                'raw_empty_rate': np.nan,
                'post_empty_rate': np.nan,
            })
coverage_df = pd.DataFrame(coverage_rows)
save_table_csv(coverage_df, 'step5_conformal_summary_v4.csv')

within_cov = coverage_df[(coverage_df['scope'] == 'within') &
    (coverage_df['method'].isin(['aps', 'lac']))].groupby(['domain', 'method',
    'alpha'], as_index=False).mean(numeric_only=True)
within_palette = {COND_LABEL[c]: COND_COLOR[c] for c in COND_ORDER}
within_cov['Domain'] = within_cov['domain'].map(COND_LABEL)
within_cov['Method'] = within_cov['method'].str.upper()
fig, axes = plt.subplots(1, 2, figsize=(IEEE_TWO_COL, 3.0),
    constrained_layout=True)
sns.lineplot(data=within_cov, x=1 - within_cov['alpha'],
    y='coverage_subject_level', hue='Domain', style='Method', markers=True,
    dashes={'APS': '', 'LAC': (3, 2)}, palette=within_palette, ax=axes[0])
axes[0].plot([0.8, 0.95], [0.8, 0.95], color='black', linestyle=':',
    linewidth=0.9)
axes[0].set_title('Within-condition empirical subject coverage')
axes[0].set_xlabel('Nominal coverage')
axes[0].set_ylabel('Empirical coverage')
axes[0].grid(alpha=0.2)
sns.lineplot(data=within_cov, x='alpha', y='post_mean_set_size', hue='Domain',
    style='Method', markers=True, dashes={'APS': '', 'LAC': (3, 2)},
    palette=within_palette, legend=False, ax=axes[1])
axes[1].set_title('Within-condition post-fallback set size')
axes[1].set_xlabel('Alpha')
axes[1].set_ylabel('Mean set size')
axes[1].grid(alpha=0.2)
if axes[0].legend_ is not None:

```

```

    axes[0].legend_.remove()
    domain_handles = [plt.Line2D([0], [0], color=within_palette[COND_LABEL[c]],
        ↳lw=1.8, label=COND_LABEL[c]) for c in COND_ORDER]
    method_handles = [
        plt.Line2D([0], [0], color='black', lw=1.5, linestyle='-', marker='o',
        ↳label='APS'),
        plt.Line2D([0], [0], color='black', lw=1.5, linestyle='--', marker='o',
        ↳label='LAC'),
        plt.Line2D([0], [0], color='black', lw=1.2, linestyle=':', label='Ideal',
        ↳nominal reference'),
    ]
    legend1 = axes[1].legend(handles=domain_handles, loc='upper left',
        ↳bbox_to_anchor=(1.02, 1.00), frameon=False, title='Domain')
    legend1._legend_box.align = 'left'
    axes[1].add_artist(legend1)
    legend2 = axes[1].legend(handles=method_handles, loc='lower left',
        ↳bbox_to_anchor=(1.02, 0.10), frameon=False, title='Method')
    legend2._legend_box.align = 'left'
    savefig_stem(fig, 'conformal_within')
    plt.show()

    cross_cov = coverage_df[(coverage_df['scope'] == 'cross') &
        ↳ (coverage_df['method'].isin(['aps', 'lac']))].groupby(['domain', 'method',
        ↳ 'alpha'], as_index=False).mean(numeric_only=True)
    dir_palette = dict(zip([DIR_LABEL[d] for d in DIRECTIONS], sns.
        ↳ color_palette('tab10', n_colors=len(DIRECTIONS))))
    cross_cov['Direction'] = cross_cov['domain'].map(DIR_LABEL)
    cross_cov['Method'] = cross_cov['method'].str.upper()
    fig, axes = plt.subplots(1, 2, figsize=(IEEE_TWO_COL, 3.0),
        ↳ constrained_layout=True)
    sns.lineplot(data=cross_cov, x=1 - cross_cov['alpha'],
        ↳ y='coverage_subject_level', hue='Direction', style='Method', markers=False,
        ↳ dashes={'APS': '', 'LAC': (3, 2)}, palette=dir_palette, ax=axes[0])
    axes[0].plot([0.8, 0.95], [0.8, 0.95], color='black', linestyle=':',
        ↳ linewidth=0.9)
    axes[0].set_title('Cross-condition empirical subject coverage')
    axes[0].set_xlabel('Nominal coverage')
    axes[0].set_ylabel('Empirical coverage')
    axes[0].grid(alpha=0.2)
    sns.lineplot(data=cross_cov, x='alpha', y='post_mean_set_size',
        ↳ hue='Direction', style='Method', markers=False, dashes={'APS': '', 'LAC':
        ↳ (3, 2)}, palette=dir_palette, legend=False, ax=axes[1])
    axes[1].set_title('Cross-condition post-fallback set size')
    axes[1].set_xlabel('Alpha')
    axes[1].set_ylabel('Mean set size')
    axes[1].grid(alpha=0.2)

```

```

if axes[0].legend_ is not None:
    axes[0].legend_.remove()
direction_handles = [plt.Line2D([0], [0], color=dir_palette[DIR_LABEL[d]], lw=1.
    ↳8, label=DIR_LABEL[d]) for d in DIRECTIONS]
method_handles = [
    plt.Line2D([0], [0], color='black', lw=1.5, linestyle='-', label='APS'),
    plt.Line2D([0], [0], color='black', lw=1.5, linestyle='--', label='LAC'),
    plt.Line2D([0], [0], color='black', lw=1.2, linestyle=':', label='Ideal
    ↳nominal reference'),
]
legend1 = axes[1].legend(handles=direction_handles, loc='upper left',
    ↳bbox_to_anchor=(1.02, 1.00), frameon=False, title='Direction')
legend1._legend_box.align = 'left'
axes[1].add_artist(legend1)
legend2 = axes[1].legend(handles=method_handles, loc='lower left',
    ↳bbox_to_anchor=(1.02, 0.0), frameon=False, title='Line style')
legend2._legend_box.align = 'left'
savefig_stem(fig, 'conformal_cross')
plt.show()

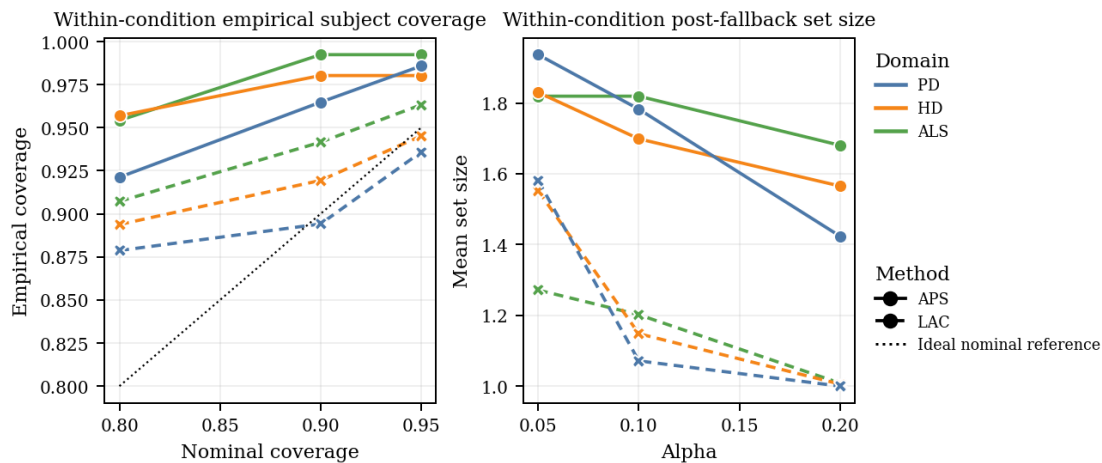
curve = coverage_df[(coverage_df['scope'] == 'within') & (coverage_df['method']
    ↳== 'hard_label_consensus_curve')].groupby(['domain', 'alpha'],
    ↳as_index=False)['coverage_subject_level'].mean().sort_values('alpha')
curve['Condition'] = curve['domain'].map(COND_LABEL)
fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.2, 2.8),
    ↳constrained_layout=True)
sns.lineplot(data=curve, x='alpha', y='coverage_subject_level',
    ↳hue='Condition', marker='o', palette=within_palette, ax=ax)
ax.set_title('hard_label_consensus_curve')
ax.set_xlabel('Aggregated strides')
ax.set_ylabel('Agreement rate')
ax.grid(alpha=0.2)
ax.legend(loc='center left', bbox_to_anchor=(1.02, 0.5), frameon=False)
savefig_stem(fig, 'step5_hard_label_consensus_curve')
plt.show()

```

Saved table: report/tables/v4/step5_conformal_summary_v4.csv

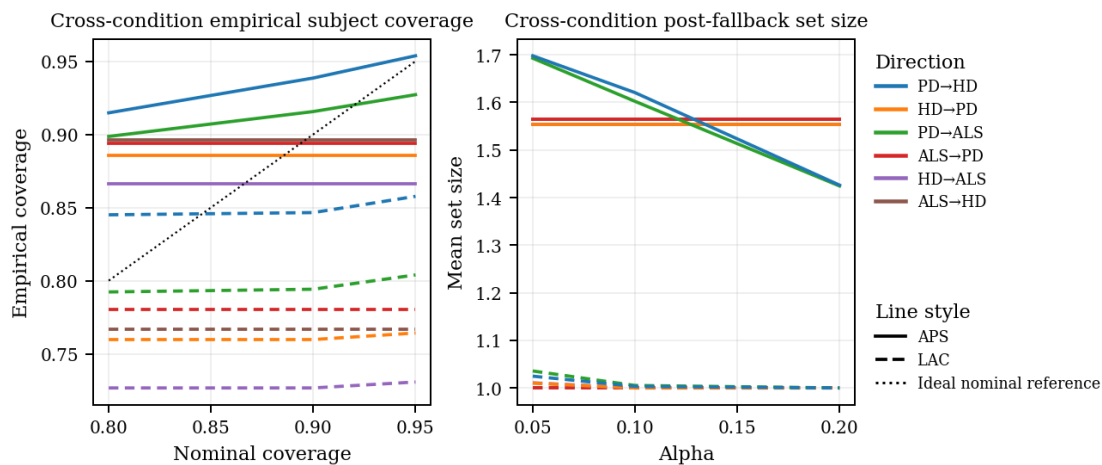
Saved figure: report/figures/v4/pdf/step5/conformal_within.pdf

Saved figure: report/figures/v4/png/step5/conformal_within.png



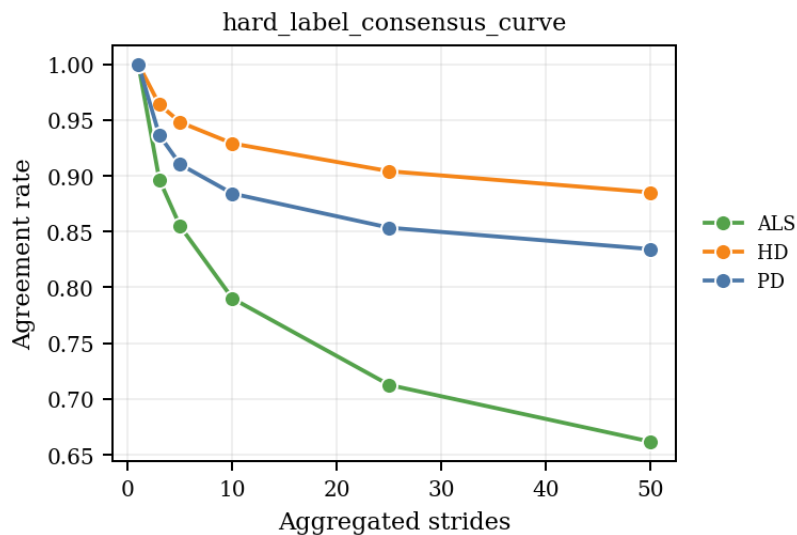
Saved figure: `report/figures/v4/pdf/step5/conformal_cross.pdf`

Saved figure: `report/figures/v4/png/step5/conformal_cross.png`



Saved figure: `report/figures/v4/pdf/step5/step5_hard_label_consensus_curve.pdf`

Saved figure: `report/figures/v4/png/step5/step5_hard_label_consensus_curve.png`



1.6 Interpretation guardrails

`gaussian_feature_space_stress_test` is a feature-space perturbation diagnostic, not a raw wearable-sensor simulation. `evaluation_row_dropout` is evaluation subsampling, not sensor dropout. `benchmark_label_corruption` is a benchmark stressor, not noisy-label training. `hard_label_consensus_curve` is a separate agreement diagnostic and should not be conflated with conformal set behavior. The strongest recurrent Step 5 story remains consistent with Step 4: **variability** and **raw timing** are the most transfer-sensitive feature families.